Khaula Molapo

20240001

Week: 6

Advanced Database

1.

In an e-commerce system, thousands of users browse products, place orders, and make payments every day. If all data is stored in one database server, it can become slow and overloaded. To solve this problem, replication and partitioning can be used.

Replication means copying the main database (primary server) to other servers called replicas. The primary server handles write operations like inserting orders, while replica servers handle read operations like viewing products. This reduces pressure on the main server and improves performance. It also increases availability because if the primary server fails, a replica can take over.

Partitioning means splitting a large table into smaller parts. For example, the orders table can be partitioned by year or by region. Orders from 2024 can be stored separately from 2025. This improves query speed because the system only searches in the relevant partition instead of the whole table.

By combining replication and partitioning, the e-commerce system becomes faster, more scalable, and more reliable.

2.

The CAP theorem states that a distributed database system can only guarantee two out of three properties: Consistency, Availability, and Partition Tolerance.

Consistency means all users see the same data at the same time. Availability means every request receives a response, even if some servers fail. Partition Tolerance means the system continues working even if there is a network failure between servers.

In real-world distributed systems, network failures can happen, so Partition Tolerance is necessary. Because of this, designers must choose between Consistency and Availability during a partition. For example, banking systems usually choose Consistency to prevent incorrect balances. Social media platforms often choose Availability to keep the system running even if some data is slightly delayed.

The CAP theorem helps database designers understand trade-offs when building distributed systems.

3.

In PostgreSQL, table partitioning can be done using declarative partitioning. For example, an orders table can be partitioned by range using the order_date column.

First, create the main table using PARTITION BY RANGE (order_date). Then create child tables for specific ranges, such as orders_2024 and orders_2025. Each partition stores data for a specific date range.

When inserting data, PostgreSQL automatically places records into the correct partition based on the date. This improves query performance because when searching for orders from 2024, the system only scans the 2024 partition.

While practicing this, I noticed that partitioning improves performance for large tables but adds complexity to database design. It requires planning and proper indexing. However, for large systems with millions of records, partitioning is very useful and improves efficiency.

4.

In Draw.io, the replication flowchart can start with Users at the top. Below that, connect to a Load Balancer. The load balancer sends write requests to the Primary Database Server and read requests to Replica Servers.

From the Primary Server, draw arrows to Replica Server 1 and Replica Server 2 to show data replication. Label the arrows as Data Sync or Replication Process.

The diagram should clearly show that the primary handles INSERT, UPDATE, and DELETE operations, while replicas handle SELECT queries. You can also add a failover arrow from Replica 1 to Primary to show that if the primary fails, a replica can become the new primary.

This diagram shows how replication improves performance, availability, and fault tolerance in distributed database systems.